



MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE
REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and
Microelectronics Department of Software and Automation Engineering

Lupasco Cristian FAF-233

Report

Laboratory work n.1

of Languages and Finite Automata

Checked by:

Cretu D., DISA, FCIM, UTM

Chişinău – 2024

Theory

A formal language can be considered to be the media or the format used to convey information from a sender entity to the one that receives it. The usual components of a language are:

1. The alphabet: Set of valid characters;
2. The vocabulary: Set of valid words;
3. The grammar: Set of rules/constraints over the lang.

Now these components can be established in an infinite amount of configurations, which actually means that whenever a language is being created, it's components should be selected in a way to make it as appropriate for it's use case as possible. Of course sometimes it is a matter of preference, that's why we ended up with lots of natural/programming/markup languages which might accomplish the same thing.

In this laboratory work, we explored the implementation of grammars and finite automata using Java. Java was chosen due to its object-oriented structure, which facilitates modeling formal languages through classes and data encapsulation. The Grammar class generated valid strings based on production rules, showcasing how recursion can simulate derivations in a formal grammar. The FiniteAutomaton class modeled state transitions and validated string acceptance, emphasizing the theoretical relationship between grammars and automata. This exercise highlighted Java's strengths in implementing abstract concepts from formal language theory and automata. Future improvements could include incorporating graphical representations of state transitions and optimizing string generation algorithms.

Conditions of the Problem

1. Implement a type/class for your grammar;
2. Add one function that would generate 5 valid strings from the language expressed by your given grammar;
3. Implement some functionality that would convert an object of type Grammar to one of type Finite Automaton
4. For the Finite Automaton, please add a method that checks if an input string can be obtained via the state transition from it

Technical Implementation

```
import java.util.*;

class Grammar {
    Set<String> VN; // Non-terminal symbols
    Set<String> VT; // Terminal symbols
    Map<String, List<String>> productions; // Production
        rules

    public Grammar(Set<String> VN, Set<String> VT, Map<
        String, List<String>> productions) {
        this.VN = VN;
        this.VT = VT;
        this.productions = productions;
    }

    public List<String> generateStrings() {
        List<String> strings = new ArrayList<>();
        Random random = new Random();
        for (int i = 0; i < 5; i++) { // Generate 5 valid
            strings
                StringBuilder sb = new StringBuilder();
                generateRecursive("S", sb, random); // Always
                    start with "S"
                strings.add(sb.toString());
            }
        return strings;
    }

    private void generateRecursive(String symbol,
        StringBuilder sb, Random random) {
        if (!VN.contains(symbol)) { // If terminal, append
            to result
            sb.append(symbol);
            return;
        }
        List<String> rules = productions.get(symbol);
```

```
        String selectedRule = rules.get(random.nextInt(
            rules.size())); // Select random rule
        for (char c : selectedRule.toCharArray()) {
            generateRecursive(String.valueOf(c), sb,
                random);
        }
    }

    public FiniteAutomaton toFiniteAutomaton() {
        return new FiniteAutomaton(VN, VT, productions);
    }
}

class FiniteAutomaton {
    Set<String> states;
    Set<String> alphabet;
    Map<String, List<String>> transitions; // Transition rules
    String startState = "S"; // Start state
    Set<String> finalStates = Set.of("C"); // Final states

    public FiniteAutomaton(Set<String> states, Set<String>
        alphabet, Map<String, List<String>> transitions) {
        this.states = states;
        this.alphabet = alphabet;
        this.transitions = transitions;
    }

    // Check if a string is accepted by the automaton
    public boolean isStringAccepted(String input) {
        return checkString(input, startState);
    }

    private boolean checkString(String input, String
        currentState) {
        // If the input is empty, check if we're in a final state
        if (input.isEmpty()) {
```

```
        return finalStates.contains(currentState);
    }

    char symbol = input.charAt(0); // First character
    // of input
    String remaining = input.substring(1); //
    // Remaining input

    // If the current state has no transitions, return
    // false
    if (!transitions.containsKey(currentState)) return
        false;

    // Check transitions from the current state
    for (String rule : transitions.get(currentState))
    {
        // If the transition matches the current
        // symbol, continue checking
        if (!rule.isEmpty() && rule.charAt(0) ==
            symbol) {
            String nextState = rule.length() > 1 ?
                rule.substring(1) : currentState; //
            // Get the next state
            if (checkString(remaining, nextState))
                return true;
        }
    }
    return false; // No valid transition found
}

}

public class GrammarAutomatonDemo {
    public static void main(String[] args) {
        // Define the grammar
        Set<String> VN = new HashSet<>(Arrays.asList("S",
            "A", "B", "C"));
        Set<String> VT = new HashSet<>(Arrays.asList("a",
            "b"));
    }
}
```

```
// Define production rules
Map<String, List<String>> productions = new
    HashMap<>();
productions.put("S", Arrays.asList("aA", "aB"));
productions.put("A", Arrays.asList("bS"));
productions.put("B", Arrays.asList("aC"));
productions.put("C", Arrays.asList("a", "bS"));

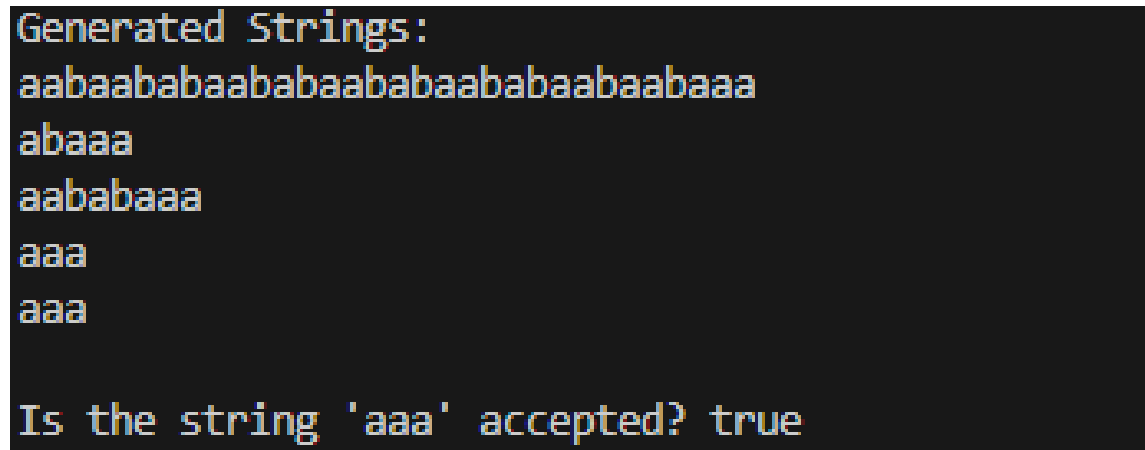
// Create Grammar object
Grammar grammar = new Grammar(VN, VT, productions)
    ;

// Generate 5 valid strings
System.out.println("Generated Strings:");
List<String> validStrings = grammar.
    generateStrings();
validStrings.forEach(System.out::println);

// Convert to finite automaton
FiniteAutomaton automaton = grammar.
    toFiniteAutomaton();

// Check if a string is accepted
String testString = "aaa"; // Example string
System.out.println("\nIs the string '" +
    testString + "' accepted?" + automaton.
    isStringAccepted(testString));
}
}
```

Visual Representation



```
Generated Strings:
aabaababaababaababaababaababaaa
abaaa
aababaaa
aaa
aaa

Is the string 'aaa' accepted? true
```

Figure 1: Screenshot from VSC

Conclusions

This laboratory work explored the implementation of grammars and finite automata. A grammar class was developed to generate valid strings using production rules, while a finite automaton class was designed to check string acceptance. The conversion from grammar to automaton was implemented, demonstrating the connection between formal languages and finite state machines. Through this exercise, key concepts of language generation, state transitions, and string validation were reinforced. Overall, this practical experience deepened understanding of formal language theory, helping to solidify its relevance in automata theory and computational linguistics. Future improvements may include more complex grammar examples and state visualizations.